

Computer Vision – Lecture 6

Segmentation as Energy Minimization

05.05.2025

Bastian Leibe

Visual Computing Institute

RWTH Aachen University

<http://www.vision.rwth-aachen.de/>

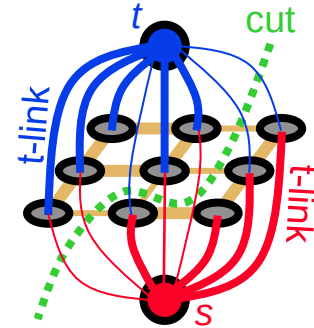
leibe@vision.rwth-aachen.de

Course Outline

- Image Processing Basics
- Segmentation
 - Segmentation as Pixel-wise Classification
 - Graph-theoretic Segmentation
- Object Recognition and Categorization
- Deep Learning for Vision
- 3D Reconstruction

Topics of This Lecture

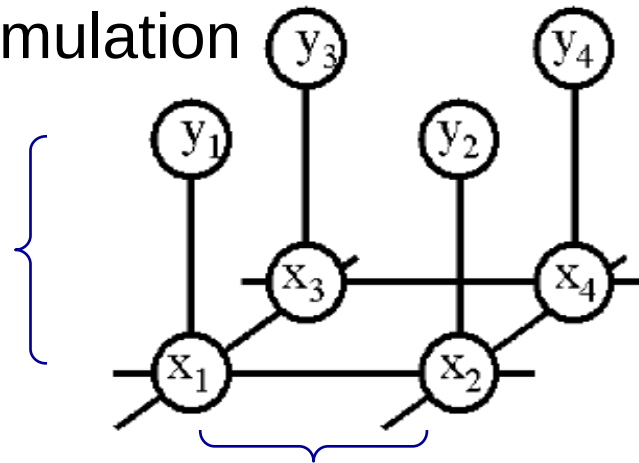
- **Recap: Segmentation as Energy Minimization**
 - Markov Random Fields
 - Energy formulation
- **Graph cuts for image segmentation**
 - Basic idea
 - s-t Mincut algorithm
 - Extension to non-binary case
- **Applications**
 - Interactive segmentation



Recap: MRFs/CRFs for Image Segmentation

- MRF/CRF formulation

Unary
potentials
 $\phi(x_i, y_i)$



Pairwise potentials
 $\psi(x_i, x_j)$

$\Rightarrow$ Minimize the energy

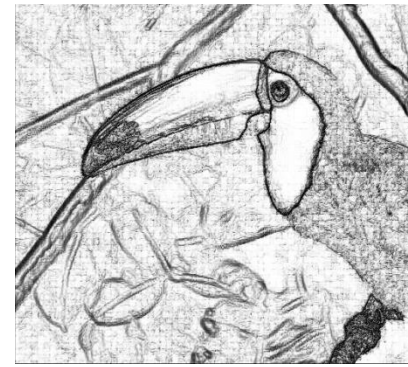
$$E(\mathbf{x}, \mathbf{y}) = \sum_i \phi(x_i, y_i) + \sum_{i,j} \psi(x_i, x_j)$$



Data (D)



Unary likelihood



Pair-wise Terms

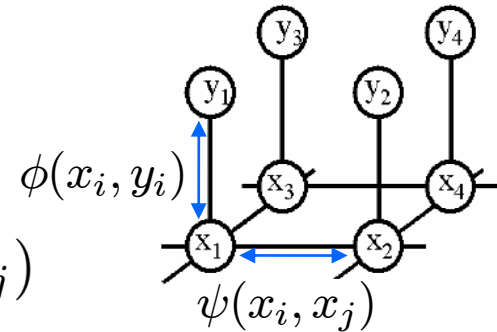


MAP Solution

Recap: Energy Formulation

- Energy function

$$E(\mathbf{x}, \mathbf{y}) = \sum_i \underbrace{\phi(x_i, y_i)}_{\text{Unary potentials}} + \sum_{i,j} \underbrace{\psi(x_i, x_j)}_{\text{Pairwise potentials}}$$

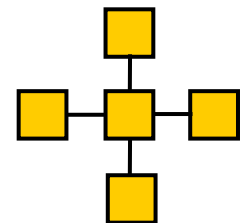


- Unary potentials ϕ

- Encode local information about the given pixel/patch
- How likely is a pixel/patch to belong to a certain class (e.g. foreground/background)?

- Pairwise potentials ψ

- Encode neighborhood information
- How different is a pixel/patch's label from that of its neighbor? (e.g. based on intensity/color/texture difference, edges)

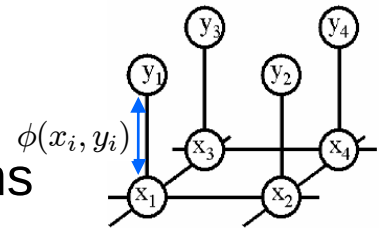


Recap: How to Set the Potentials?

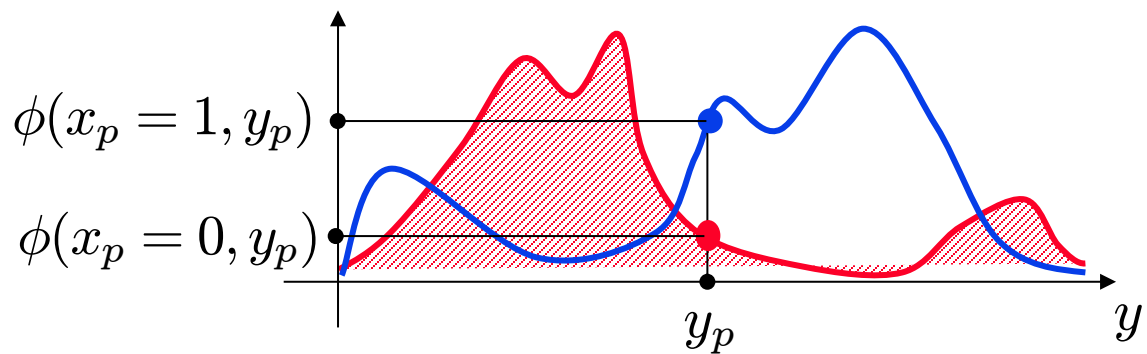
- Unary potentials

- E.g., color model, modeled with a Mixture of Gaussians

$$\phi(x_i, y_i) = -\theta_\phi \log \sum_k \pi_{k, x_i} \mathcal{N}(y_i; \boldsymbol{\mu}_{k, x_i}, \boldsymbol{\Sigma}_{k, x_i})$$



⇒ Learn color distributions for each label



Recap: How to Set the Potentials?

- Pairwise potentials

- Simplest choice: “Potts model”

$$\psi(x_i, x_j) = \theta_\psi \delta(x_i \neq x_j)$$

- Effect: Fixed penalty for all label changes

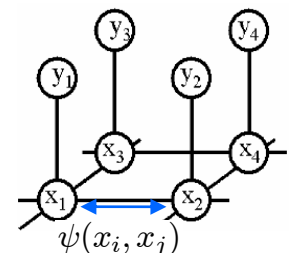
- Extension: “contrast-sensitive Potts model”

$$\psi(x_i, x_j, g_{ij}(\mathbf{y})) = \theta_\psi g_{ij}(\mathbf{y}) \delta(x_i \neq x_j)$$

where

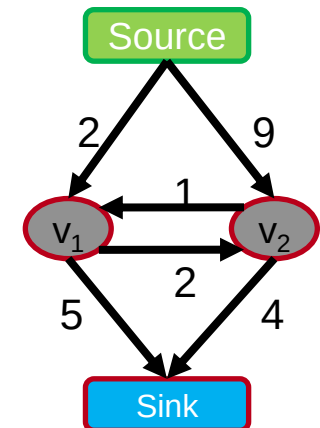
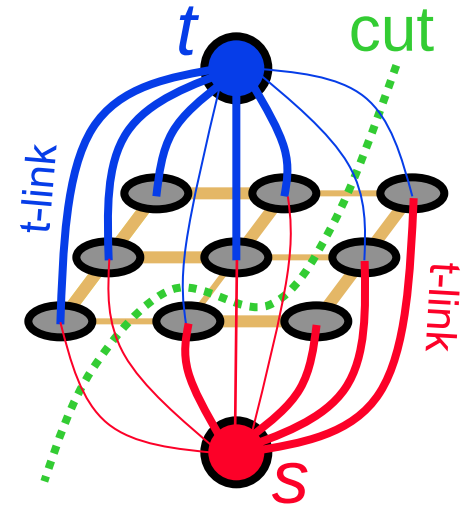
$$g_{ij}(\mathbf{y}) = e^{-\beta \|y_i - y_j\|^2} \quad \beta = 2 \text{ avg} (\|y_i - y_j\|^2)$$

- Effect: Discourages label changes except in places where there is also a large change in the observations.



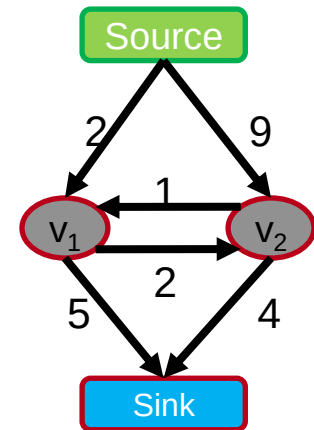
Recap: Graph-Cuts Energy Minimization

- Solve an equivalent graph cut problem
 1. Introduce extra nodes: **source** and **sink**
 2. Weight connections to source/sink (t-links) by $\phi(x_i = s)$ and $\phi(x_i = t)$, respectively.
 3. Weight connections between nodes (n-links) by $\psi(x_i, x_j)$.
 4. Find the minimum cost cut that separates source from sink.
 $\Rightarrow$ Solution is equivalent to minimum of the energy.
- s-t Mincut can be solved efficiently
 - Dual to the well-known max flow problem
 - Very efficient algorithms available for regular grid graphs (several MPixels/s)
 - **Globally optimal result** for 2-class problems

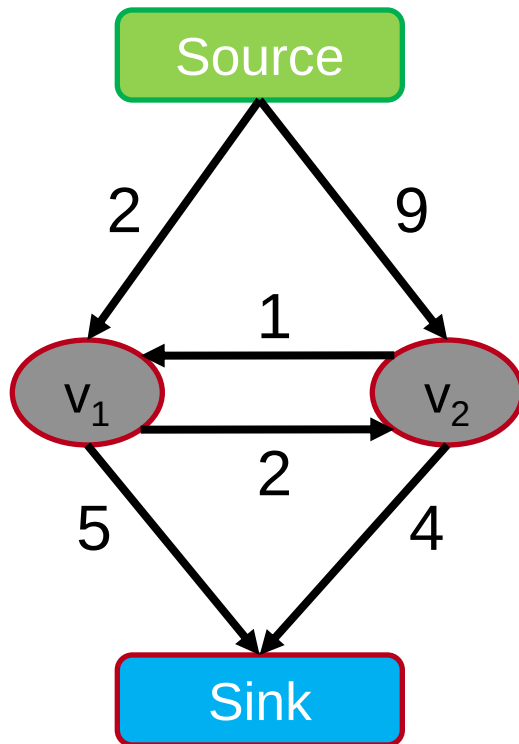


Topics of This Lecture

- Segmentation as Energy Minimization
 - Markov Random Fields
 - Energy formulation
- Graph cuts for image segmentation
 - Basic idea
 - s-t Mincut algorithm
 - Extension to non-binary case
- Applications
 - Interactive segmentation



How Does it Work? The s-t-Mincut Problem



Graph (V, E, C)

Vertices $V = \{v_1, v_2 \dots v_n\}$

Edges $E = \{(v_1, v_2) \dots\}$

Costs $C = \{c_{(1, 2)} \dots\}$

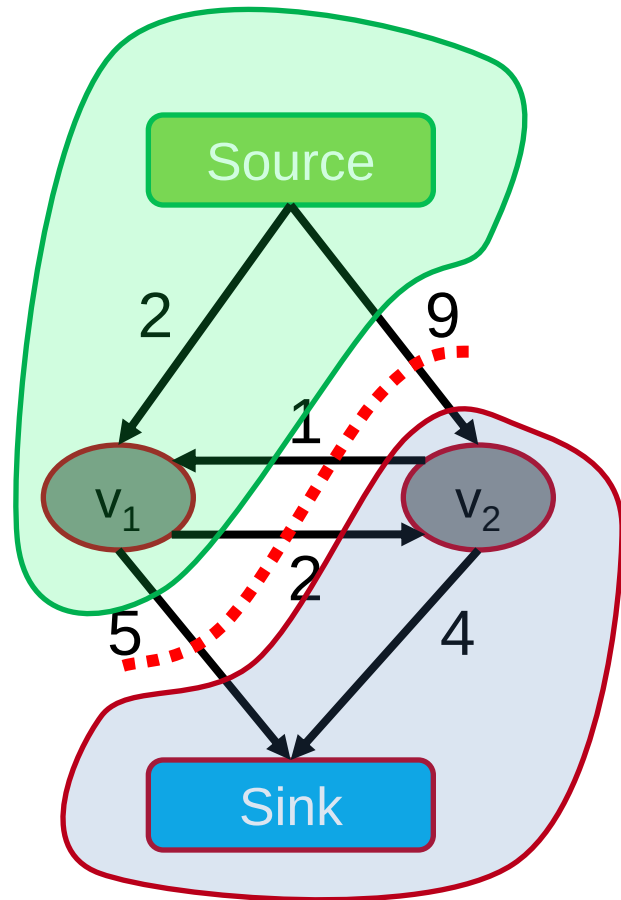
The s-t-Mincut Problem

What is an st-cut?

An st-cut (S, T) divides the nodes between source and sink.

What is the cost of a st-cut?

Sum of cost of all edges going from S to T



$$5 + 2 + 9 = 16$$

The s-t-Mincut Problem

What is an st-cut?

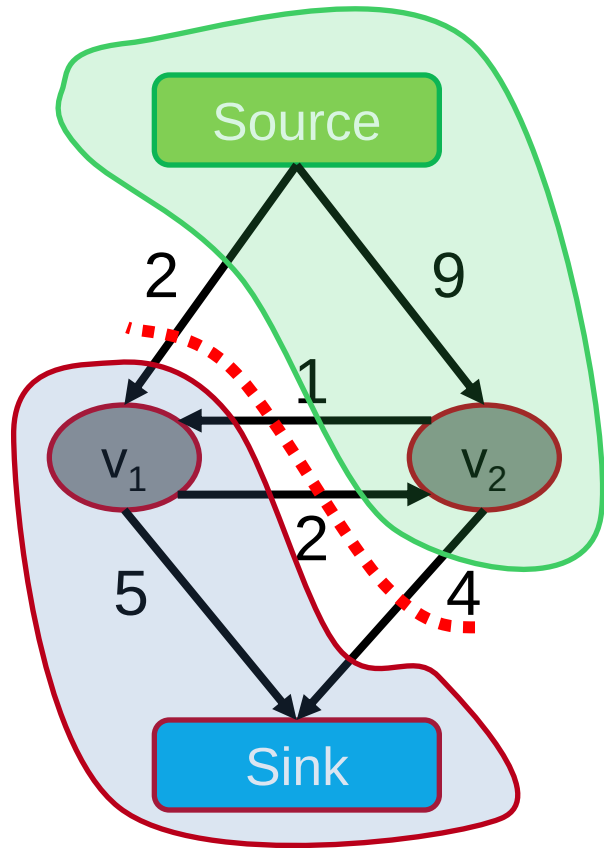
An st-cut (S, T) divides the nodes between source and sink.

What is the cost of a st-cut?

Sum of cost of all edges going from S to T

What is the st-mincut?

st-cut with the minimum cost

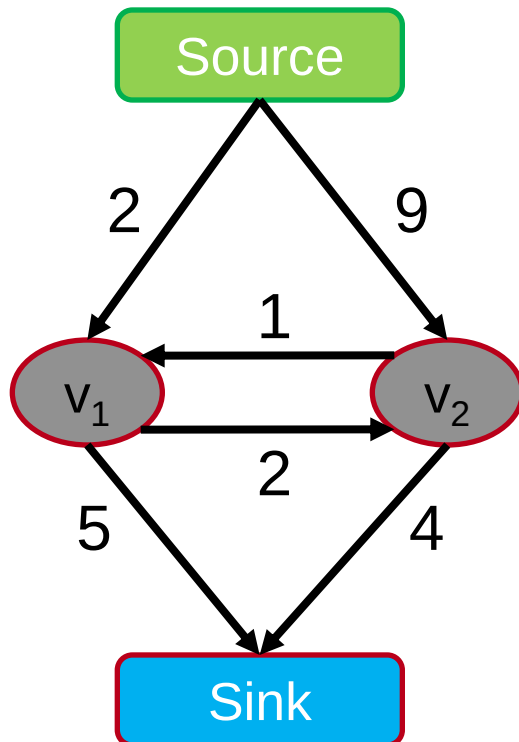


$$2 + 1 + 4 = 7$$

How to Compute the s-t-Mincut?

Solve the dual maximum flow problem

Compute the maximum flow between
Source and Sink



Constraints

Edges: $\text{Flow} < \text{Capacity}$

Nodes: $\text{Flow in} = \text{Flow out}$

Min-cut/Max-flow Theorem

In every network, the maximum flow equals the cost of the st-mincut

History of Maxflow Algorithms

Augmenting Path and Push-Relabel

year	discoverer(s)	bound
1951	Dantzig	$O(n^2 m U)$
1955	Ford & Fulkerson	$O(m^2 U)$
1970	Dinitz	$O(n^2 m)$
1972	Edmonds & Karp	$O(m^2 \log U)$
1973	Dinitz	$O(nm \log U)$
1974	Karzanov	$O(n^3)$
1977	Cherkassky	$O(n^2 m^{1/2})$
1980	Galil & Naamad	$O(nm \log^2 n)$
1983	Sleator & Tarjan	$O(nm \log n)$
1986	Goldberg & Tarjan	$O(nm \log(n^2/m))$
1987	Ahuja & Orlin	$O(nm + n^2 \log U)$
1987	Ahuja et al.	$O(nm \log(n \sqrt{\log U}/m))$
1989	Cheriyani & Hagerup	$E(nm + n^2 \log^2 n)$
1990	Cheriyani et al.	$O(n^3 / \log n)$
1990	Alon	$O(nm + n^{8/3} \log n)$
1992	King et al.	$O(nm + n^{2+\epsilon})$
1993	Phillips & Westbrook	$O(nm(\log_{m/n} n + \log^{2+\epsilon} n))$
1994	King et al.	$O(nm \log_{m/(n \log n)} n)$
1997	Goldberg & Rao	$O(m^{3/2} \log(n^2/m) \log U)$ $O(n^{2/3} m \log(n^2/m) \log U)$

n : #nodes

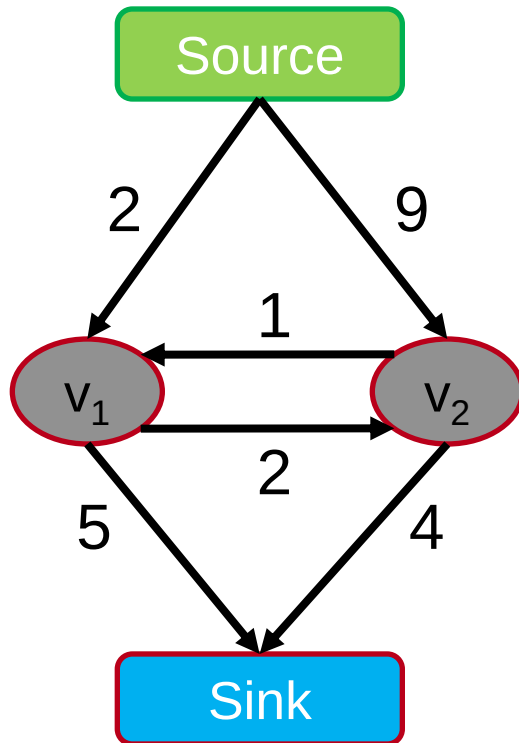
m : #edges

U : maximum
edge weight

Algorithms
assume non-
negative edge
weights

Maxflow Algorithms

Flow = 0



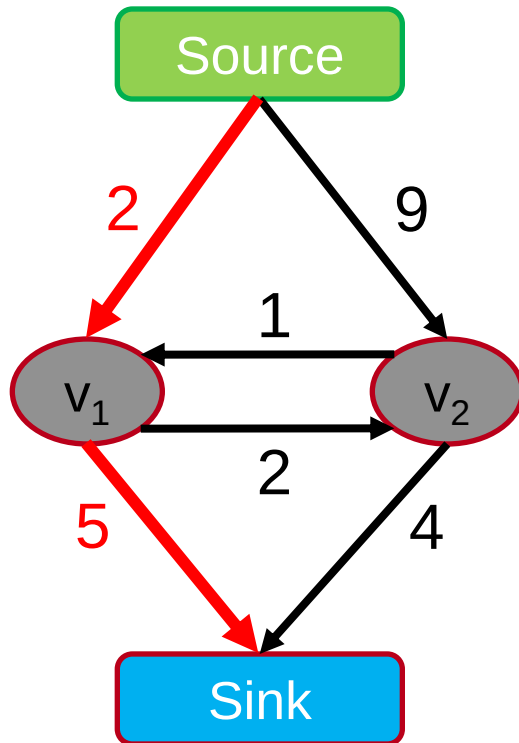
Augmenting Path Based Algorithms

1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges
4. Repeat until no path can be found

Algorithms assume non-negative capacity

Maxflow Algorithms

Flow = 0



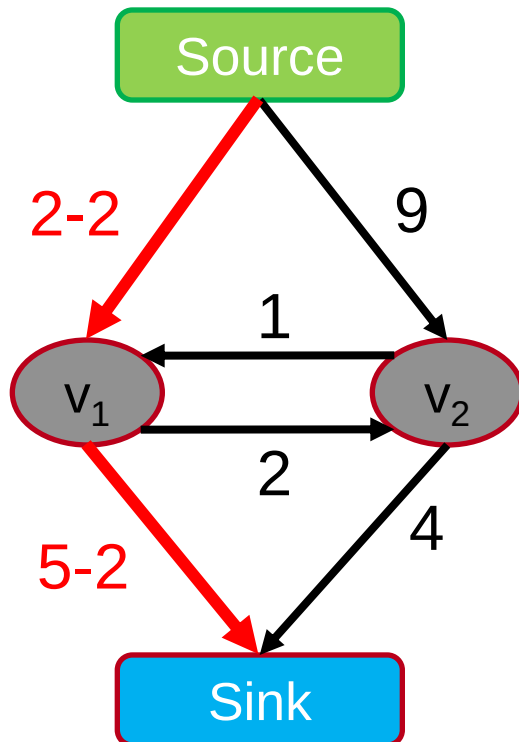
Augmenting Path Based Algorithms

1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges
4. Repeat until no path can be found

Algorithms assume non-negative capacity

Maxflow Algorithms

Flow = 0 + 2



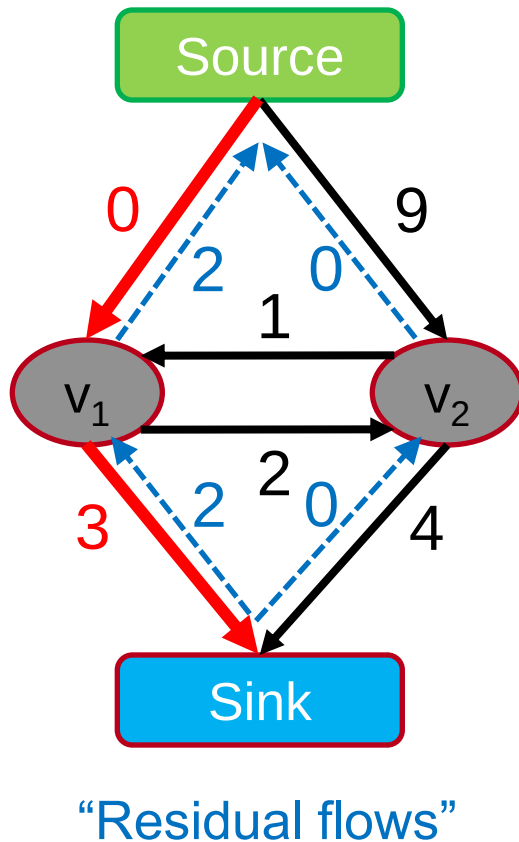
Augmenting Path Based Algorithms

1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges
4. Repeat until no path can be found

Algorithms assume non-negative capacity

Maxflow Algorithms

Flow = 2



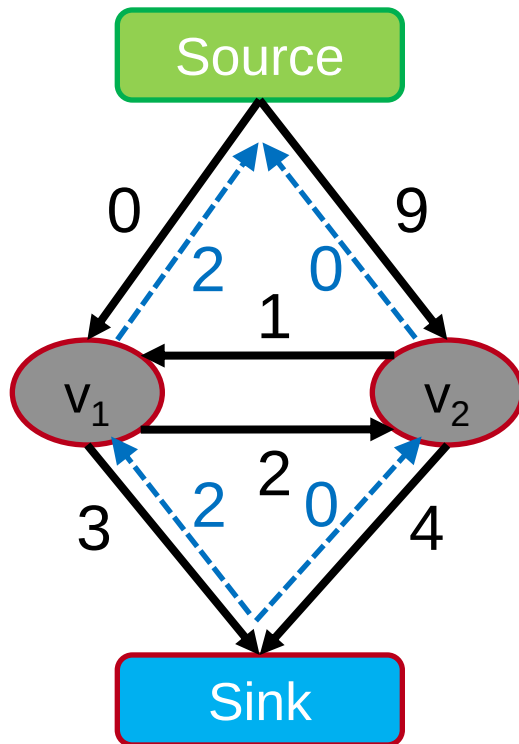
Augmenting Path Based Algorithms

1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges and record "residual flows"
4. Repeat until no path can be found

Algorithms assume non-negative capacity

Maxflow Algorithms

Flow = 2



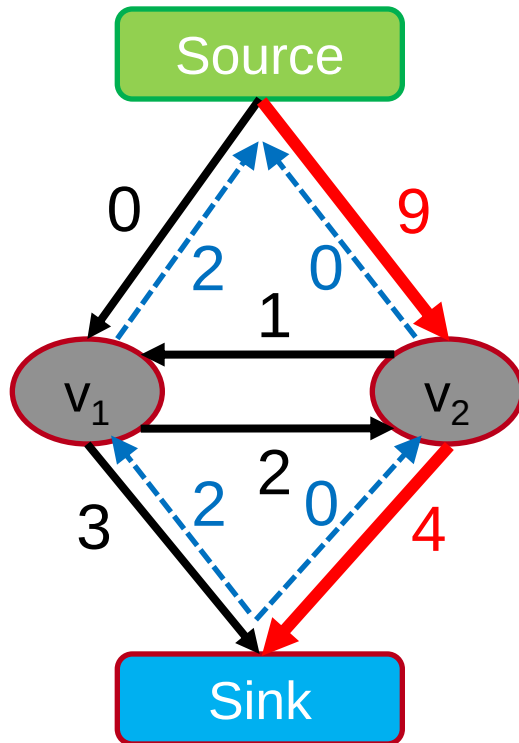
Augmenting Path Based Algorithms

1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges
4. Repeat until no path can be found

Algorithms assume non-negative capacity

Maxflow Algorithms

Flow = 2



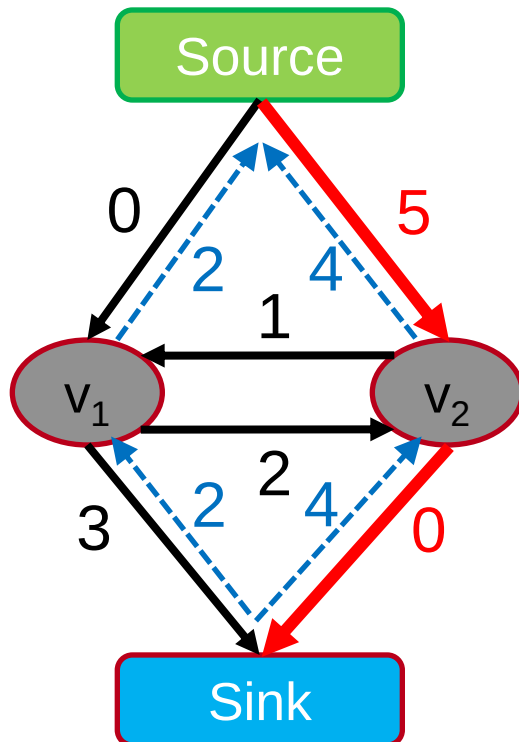
Augmenting Path Based Algorithms

1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges
4. Repeat until no path can be found

Algorithms assume non-negative capacity

Maxflow Algorithms

$$\text{Flow} = 2 + 4$$



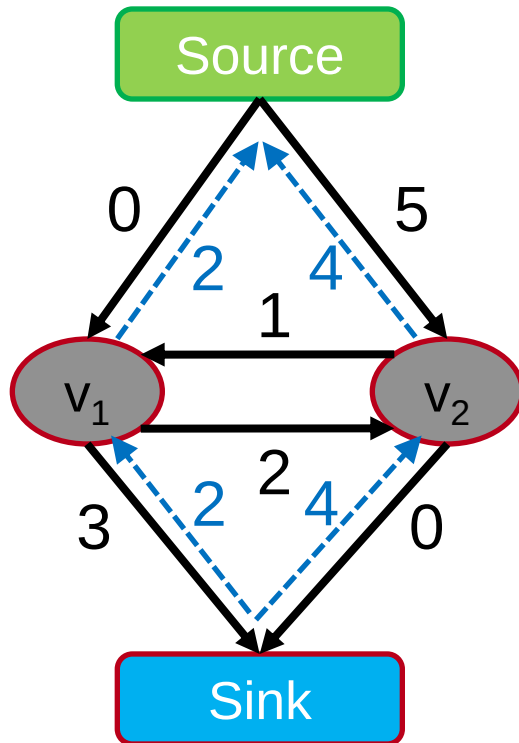
Augmenting Path Based Algorithms

1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges
4. Repeat until no path can be found

Algorithms assume non-negative capacity

Maxflow Algorithms

Flow = 6



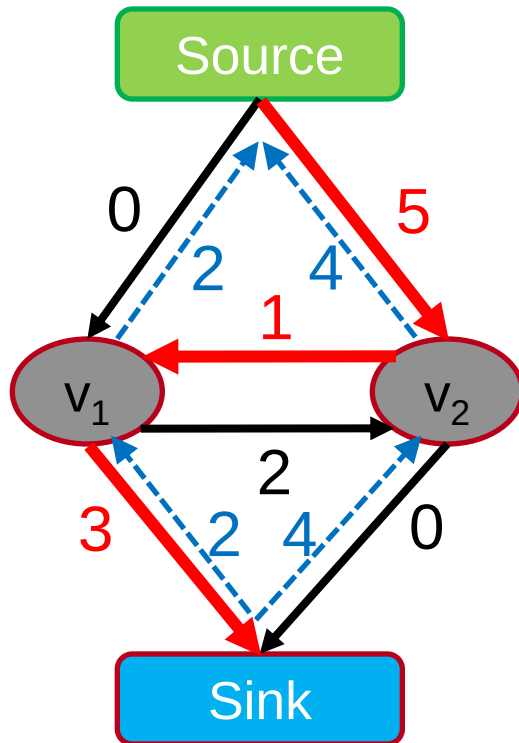
Augmenting Path Based Algorithms

1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges
4. Repeat until no path can be found

Algorithms assume non-negative capacity

Maxflow Algorithms

Flow = 6



Augmenting Path Based Algorithms

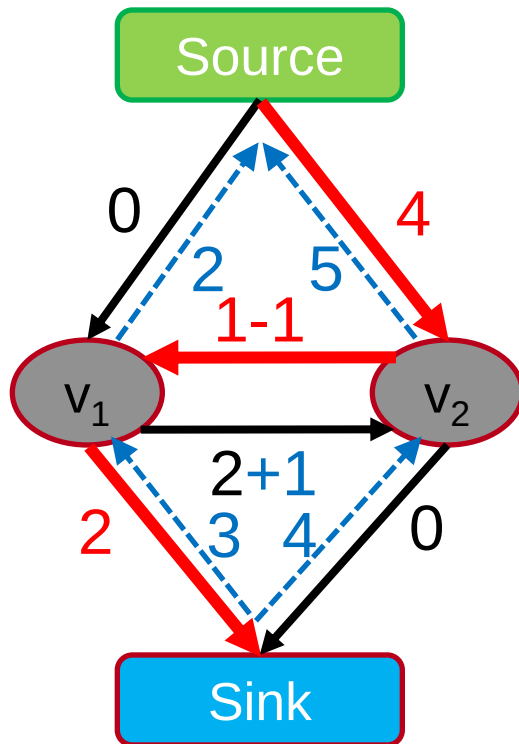
1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges
4. Repeat until no path can be found

Algorithms assume non-negative capacity

Maxflow Algorithms

Flow = 6 + 1

Augmenting Path Based Algorithms

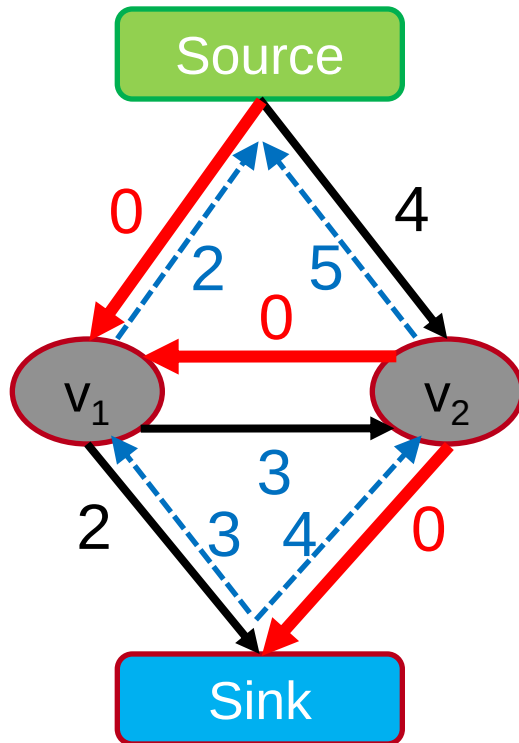


1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges
4. Repeat until no path can be found

Algorithms assume non-negative capacity

Maxflow Algorithms

Flow = 7



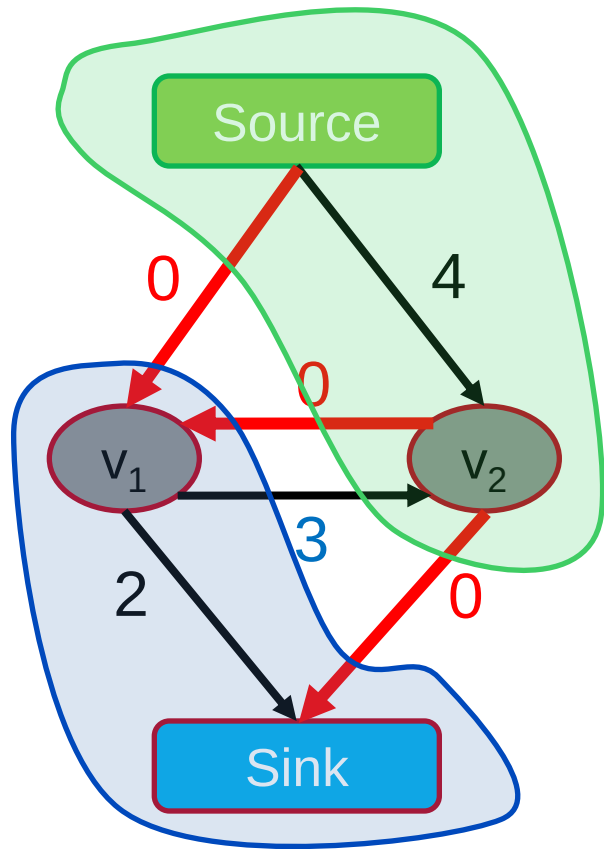
Augmenting Path Based Algorithms

1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges
4. Repeat until **no path can be found**

Algorithms assume non-negative capacity

Maxflow Algorithms

Flow = 7



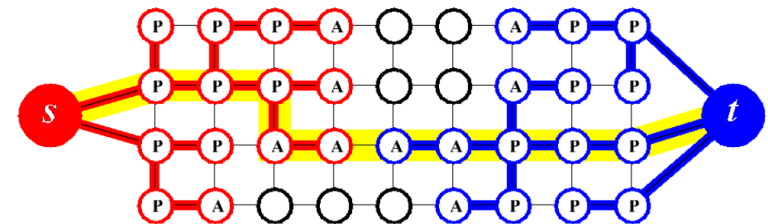
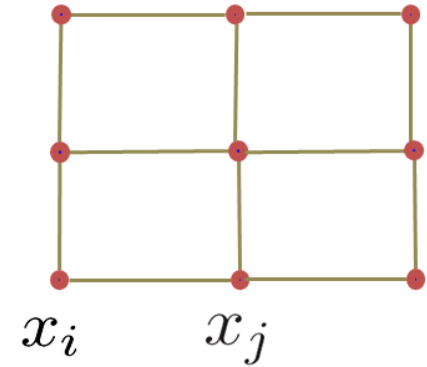
Augmenting Path Based Algorithms

1. Find path from source to sink with positive capacity
2. Push maximum possible flow through this path
3. Adjust the capacity of the used edges
4. Repeat until no path can be found

Algorithms assume non-negative capacity

Applications: Maxflow in Computer Vision

- Specialized algorithms for vision problems
 - Grid graphs
 - Low connectivity ($m \sim O(n)$)
- Dual search tree augmenting path algorithm [Boykov and Kolmogorov PAMI 2004]
 - Finds approximate shortest augmenting paths efficiently.
 - High worst-case time complexity.
 - Empirically outperforms other algorithms on vision problems.
 - Efficient code available on the web <http://pub.ist.ac.at/~vnk/software.html>

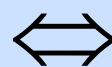


When Can s-t Graph Cuts Be Applied?

$$E(L) = \sum_{\substack{p \\ \text{t-links}}} \overset{\text{Unary potentials}}{E_p(L_p)} + \sum_{\substack{pq \in N \\ \text{n-links}}} \overset{\text{Pairwise potentials}}{E(L_p, L_q)} \quad L_p \in \{s, t\}$$

- s-t graph cuts can only globally minimize **binary energies** that are **submodular**. [Boros & Hummer, 2002, Kolmogorov & Zabih, 2004]

**$E(L)$ can be minimized
by s-t graph cuts**



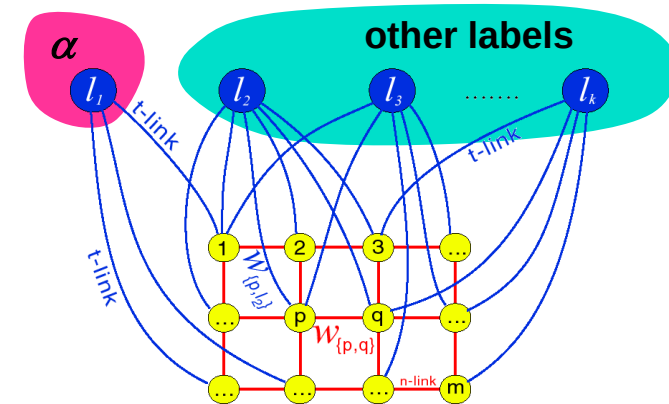
$$E(s, s) + E(t, t) \leq E(s, t) + E(t, s)$$

Submodularity (“convexity”)

- Submodularity is the discrete equivalent to convexity.
 - Implies that every local energy minimum is a global minimum.
 - ⇒ Solution will be globally optimal.

Topics of This Lecture

- Recap: Segmentation as Energy Minimization
 - Markov Random Fields
 - Energy formulation
- Graph cuts for image segmentation
 - Basic idea
 - s-t Mincut algorithm
 - Extension to non-binary case
- Applications
 - Interactive segmentation

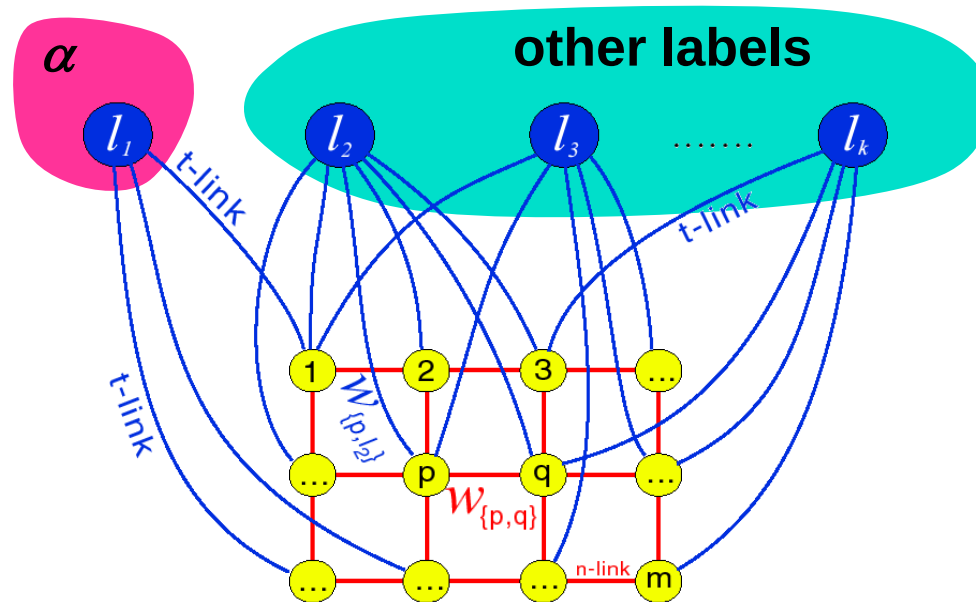


Dealing with Non-Binary Cases

- Limitation to binary energies is often a nuisance.
⇒ E.g. binary segmentation only...
- We would like to solve also multi-label problems.
 - The bad news: Problem is NP-hard with 3 or more labels!
- There exist some approximation algorithms which extend graph cuts to the multi-label case:
 - α -Expansion
 - $\alpha\beta$ -Swap
- They are no longer guaranteed to return the globally optimal result.
 - But α -Expansion has a guaranteed approximation quality (2-approx) and converges in a few iterations.

α -Expansion Move

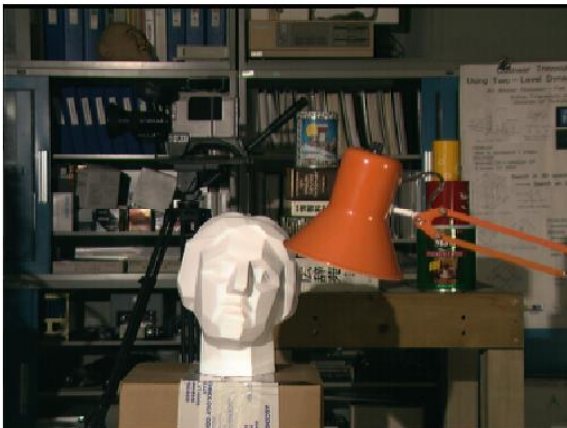
- Basic idea:
 - Break multi-way cut computation into a sequence of binary s-t cuts.



α -Expansion Algorithm

1. Start with any initial solution
2. For each label “ α ” in any (e.g. random) order:
 1. Compute optimal α -expansion move (s-t graph cuts).
 2. Decline the move if there is no energy decrease.
3. Stop when no expansion move would decrease energy.

Example: Stereo Vision



Original pair of “stereo” images



Depth map

α -Expansion Moves

- In each α -expansion a given label “ α ” grabs space from other labels



initial solution

● -expansion

● -expansion

● -expansion

● -expansion

● -expansion

● -expansion

● -expansion

For each move, we choose the expansion that gives the largest decrease in the energy: $\Rightarrow$ binary optimization problem

Topics of This Lecture

- Recap: Segmentation as Energy Minimization
 - Markov Random Fields
 - Energy formulation
- Graph cuts for image segmentation
 - Basic idea
 - s-t Mincut algorithm
 - Extension to non-binary case
- Applications
 - Interactive segmentation



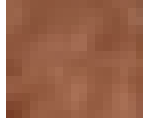
GraphCut Applications: “GrabCut”

- Interactive Image Segmentation [Boykov & Jolly, ICCV’01]
 - Rough region cues sufficient
 - Segmentation boundary can be extracted from edges
- Procedure
 - User marks foreground and background regions with a brush.
 - This is used to create an initial segmentation which can then be corrected by additional brush strokes.

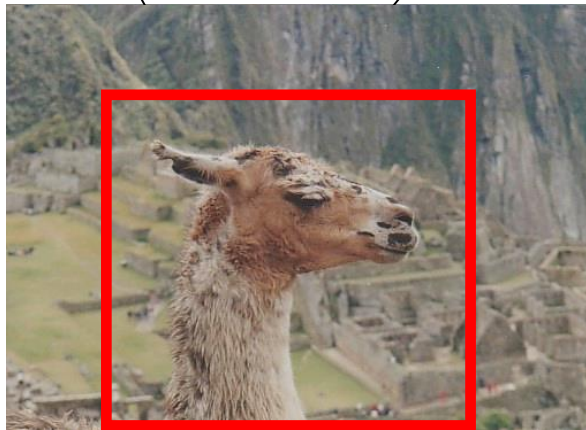


GrabCut: Data Model

Foreground
color



Background
color



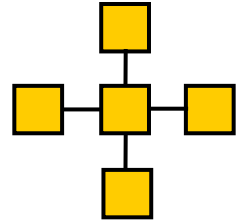
Global optimum of the
energy

- Obtained from interactive user input
 - User marks foreground and background regions with a brush
 - Alternatively, user can specify a bounding box

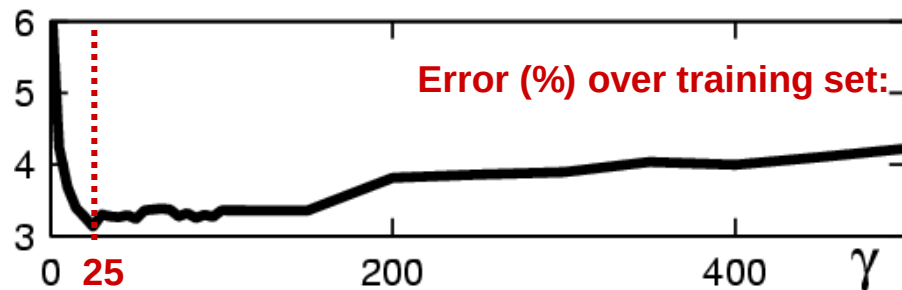
GrabCut: Coherence Model

- Pairwise potentials: Contrast-sensitive Potts Model

$$\psi(x, y) = \gamma \sum_{(m,n) \in C} \delta[x_n \neq x_m] e^{-\beta \|y_m - y_n\|^2}$$



How to choose γ ?

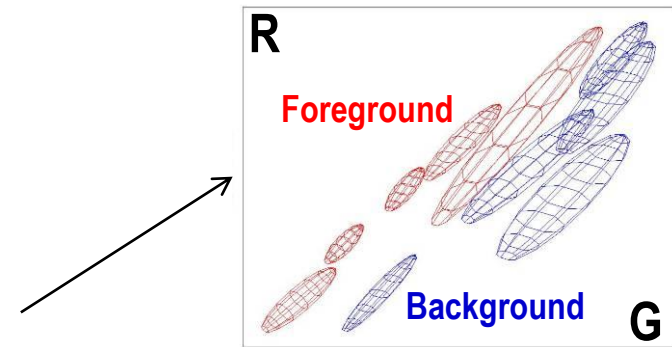


B. Leibe

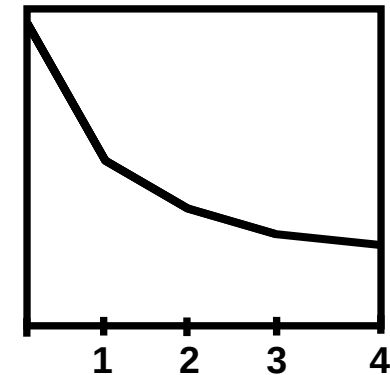
Iterated Graph Cuts



Result



Color model
(Mixture of Gaussians)



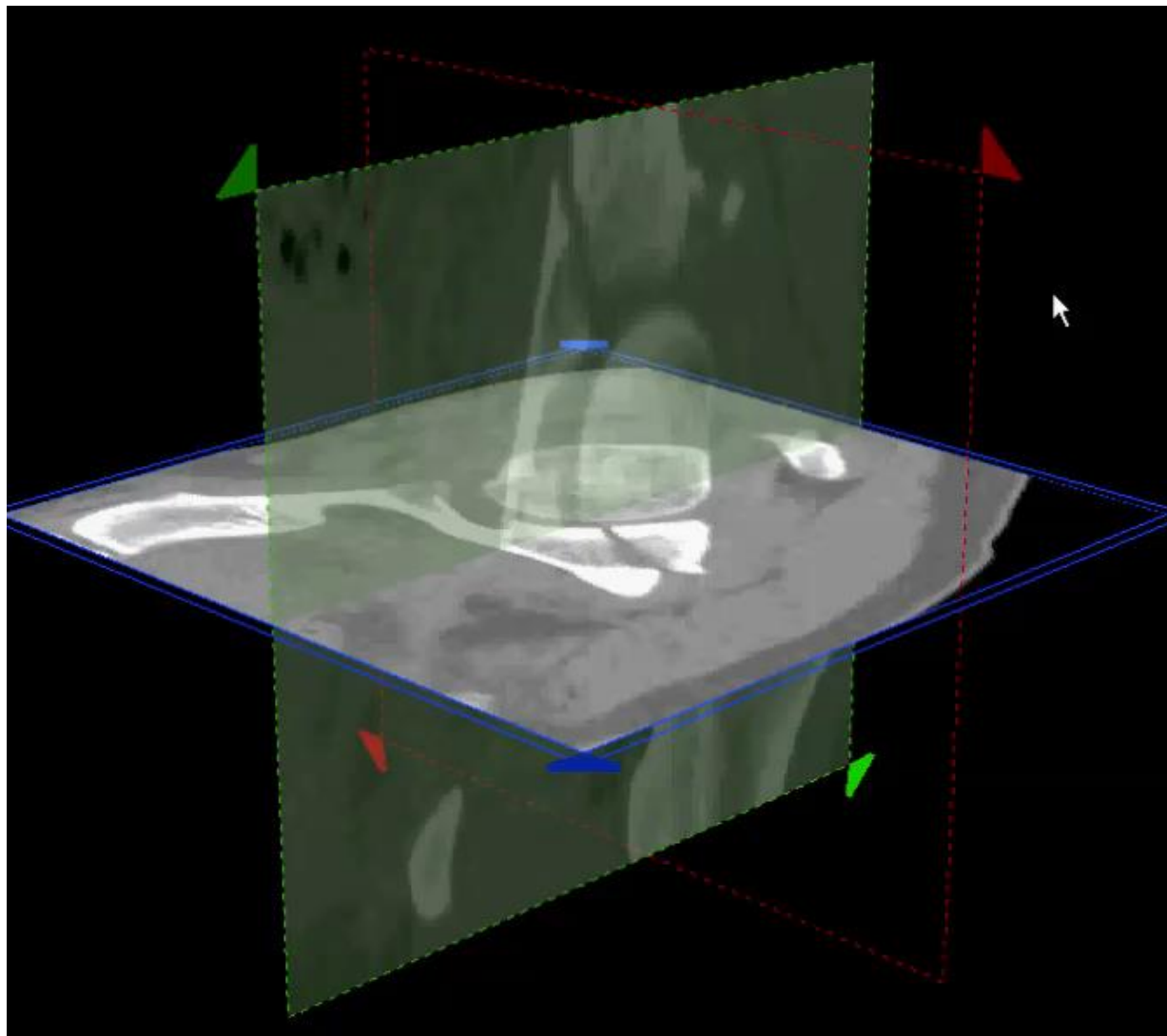
Energy after
each iteration

GrabCut: Example Results



- *This is included in all MS Office versions since 2010!*

Applications: Interactive 3D Segmentation



Summary: Graph Cuts Segmentation

- Pros

- Powerful technique, based on probabilistic model (MRF).
- Applicable for a wide range of problems.
- Very efficient algorithms available for vision problems.
- For a long time the de-facto standard for many segmentation tasks.

- Cons/Issues

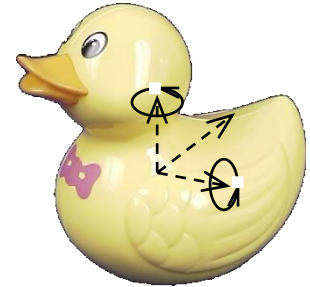
- Graph cuts can only solve a limited class of models
 - Submodular energy functions
 - Can capture only part of the expressiveness of MRFs
- Only approximate algorithms available for multi-label case

Course Outline

- Image Processing Basics
- Segmentation
 - Segmentation as Pixel-wise Classification
 - Graph-theoretic Segmentation
- Object Recognition and Categorization
- Deep Learning for Vision
- 3D Reconstruction

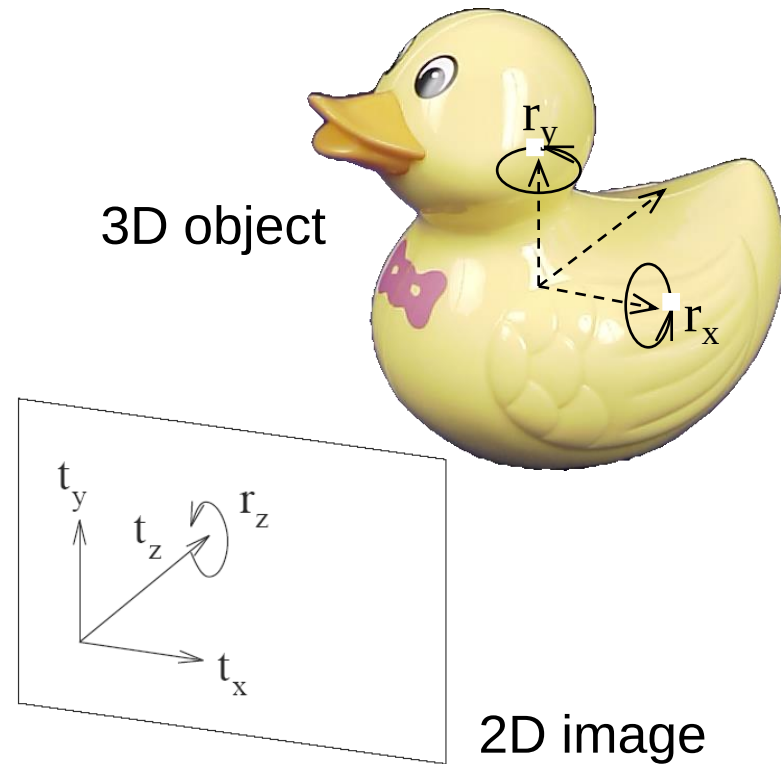
Topics of This Lecture

- Object Recognition and Categorization
 - Problem Definitions
 - Challenges
- Sliding-Window based Object Detection
 - Detection via Classification
 - Global Representations
 - Classifier Construction



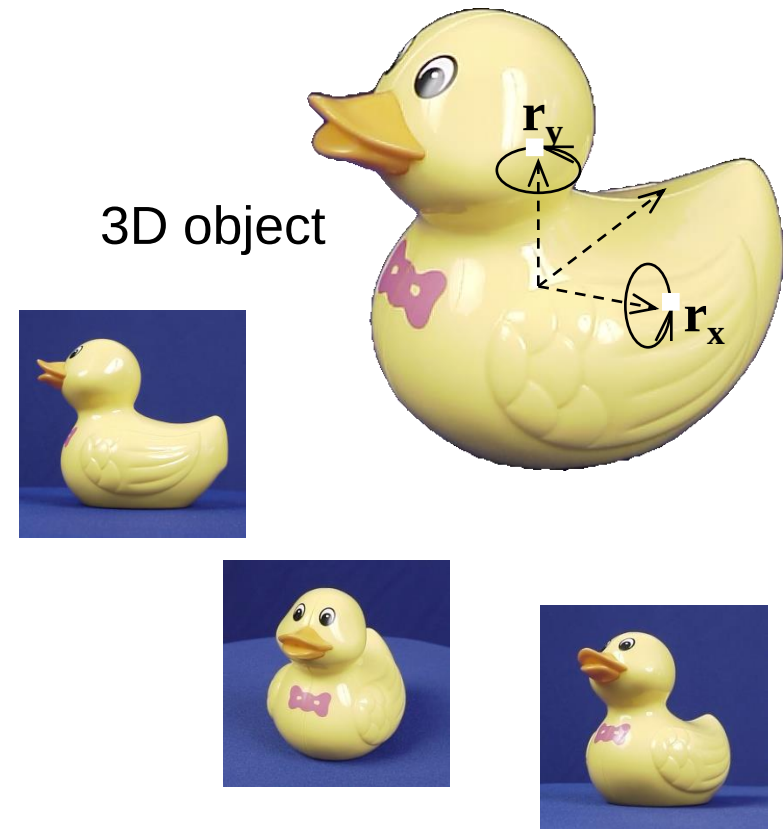
Object Recognition: Challenges

- Viewpoint changes
 - Translation
 - Image-plane rotation
 - Scale changes
 - Out-of-plane rotation
- Illumination
- Noise
- Clutter
- Occlusion



Appearance-Based Recognition

- Basic assumption
 - Objects can be represented by a set of images (“appearances”).
 - For recognition, it is sufficient to just compare the 2D appearances.
 - No 3D model is needed.

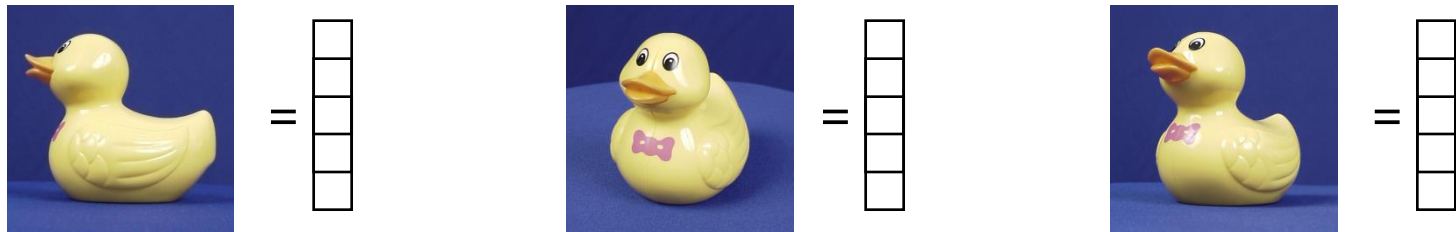


⇒ Fundamental paradigm shift in the 90's

Global Representation

- Idea

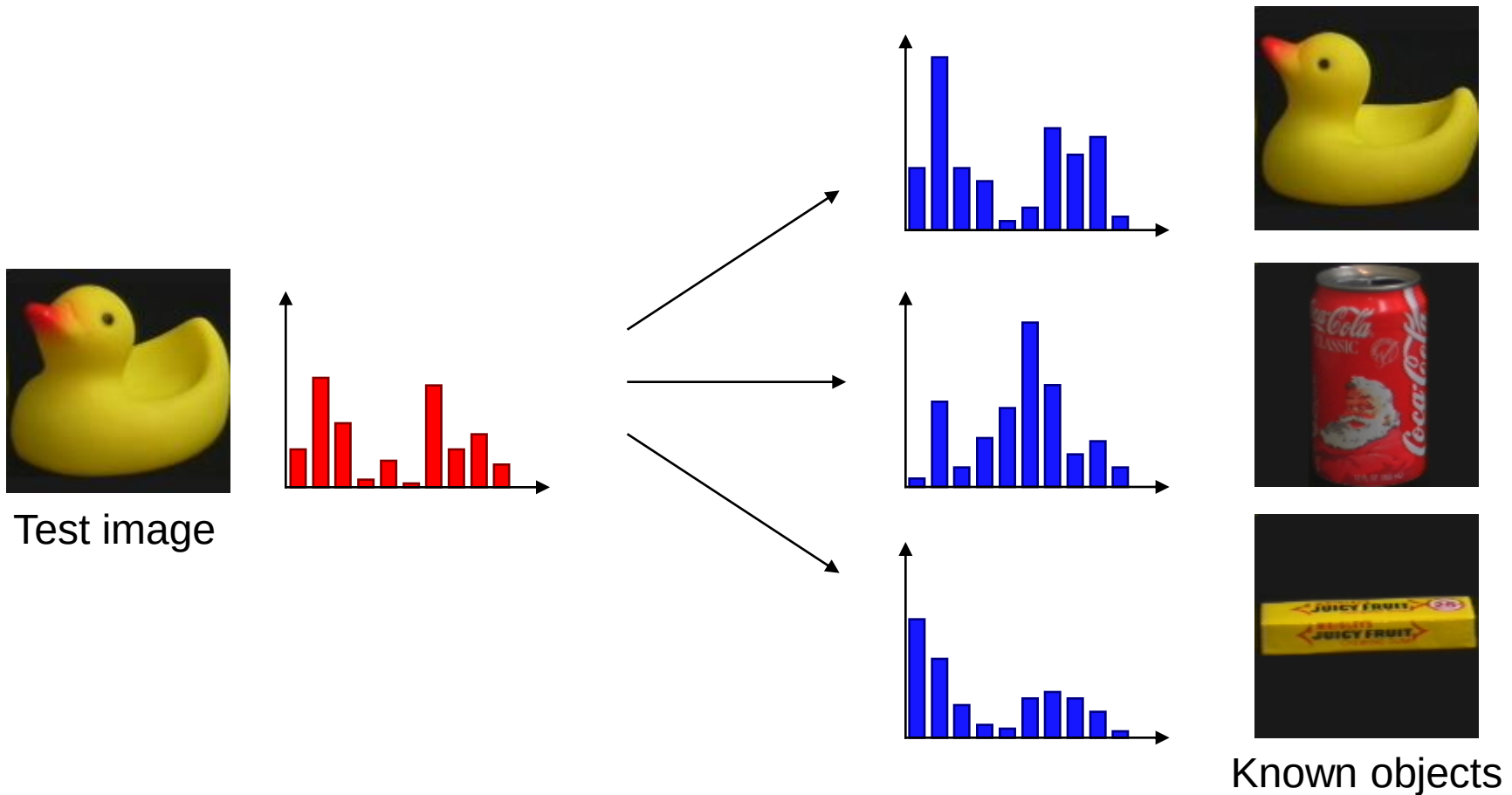
- Represent each object (view) by a global descriptor (=feature vector)



- For recognizing objects, just match the descriptors.
- Some modes of variation are built into the descriptor, the others have to be incorporated in the training data.
 - E.g., a descriptor can be made invariant to image-plane rotations.
 - Other variations:
 - Viewpoint changes
 - Translation
 - Scale changes
 - Out-of-plane rotation
 - Illumination
 - Noise
 - Clutter
 - Occlusion

Appearance based Recognition

- Recognition as feature vector matching

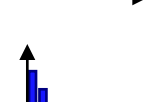
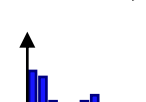
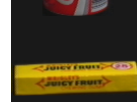
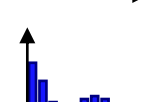
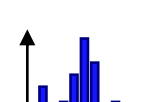
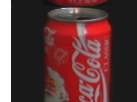
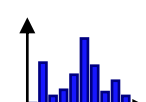
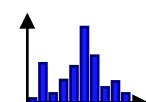
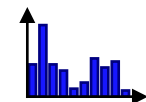
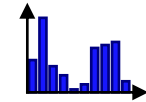
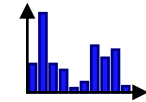
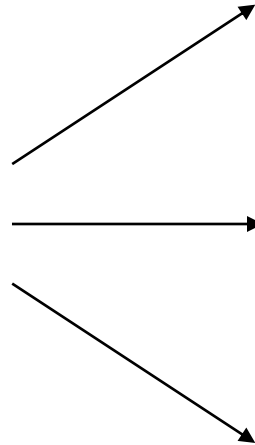
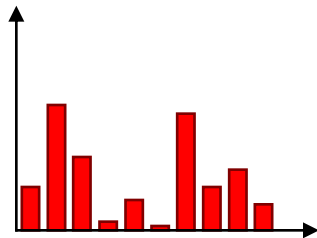


Appearance based Recognition

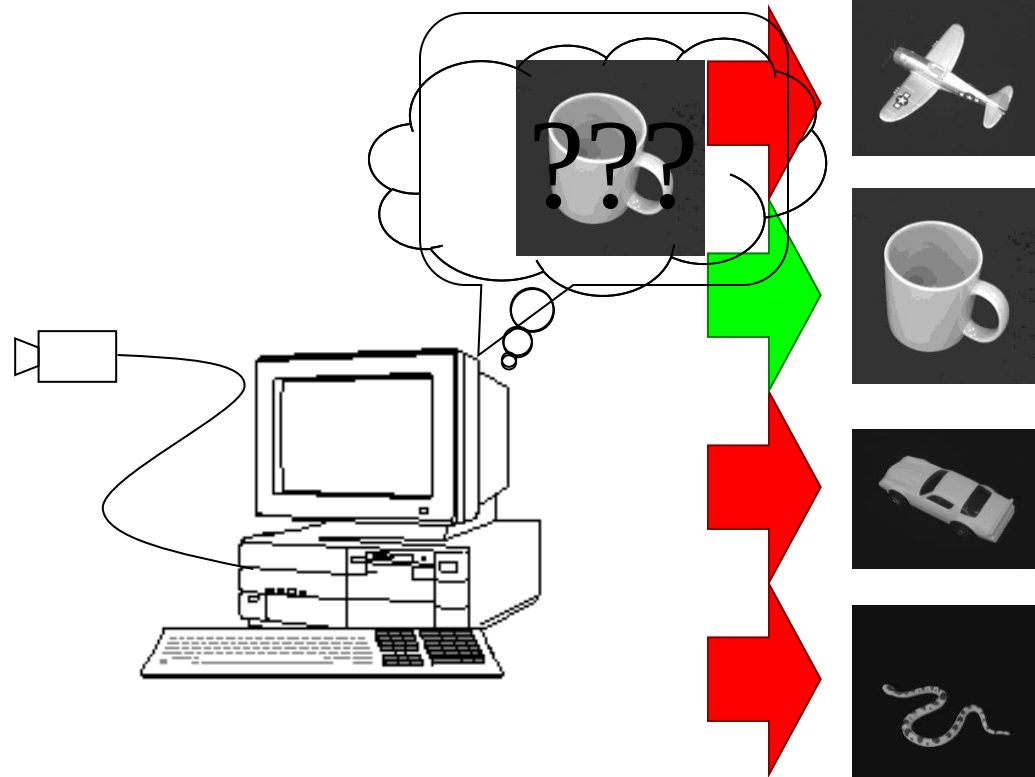
- With multiple training views



Test image



Identification vs. Categorization

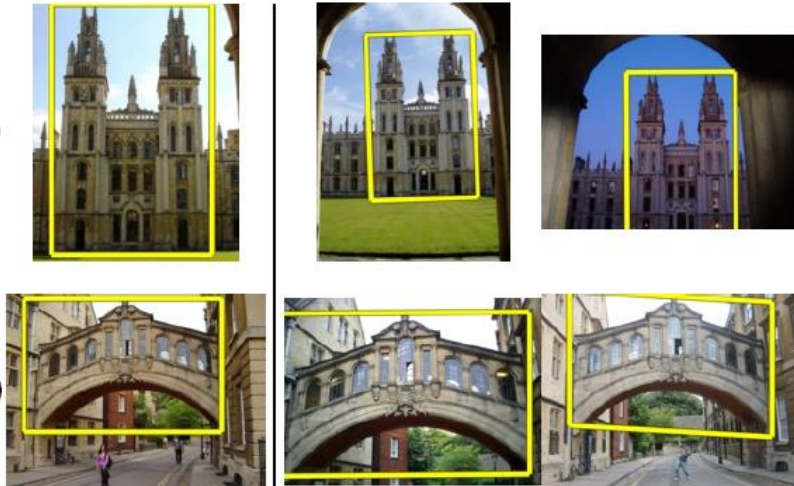


Identification vs. Categorization

- Find *this particular* object



- Recognize ANY car



- Recognize ANY cow



Object Categorization – Potential Applications

There is a wide range of applications, including...



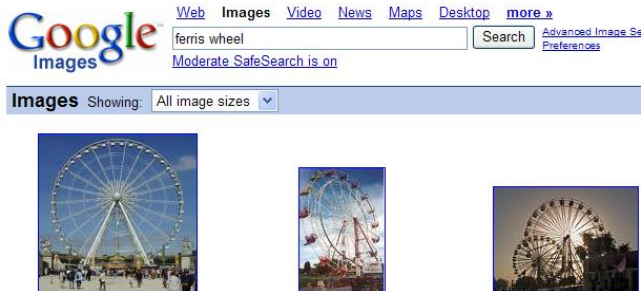
Autonomous robots



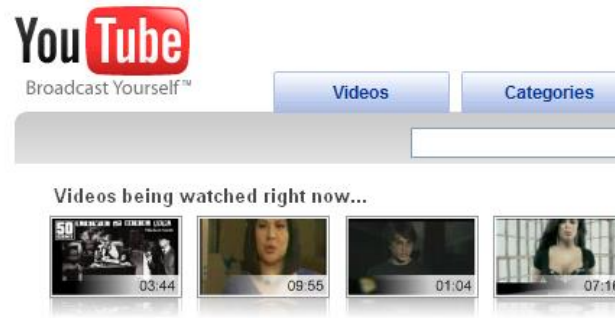
Navigation, driver safety



Consumer electronics



Content-based retrieval and analysis for
images and videos



Medical image
analysis

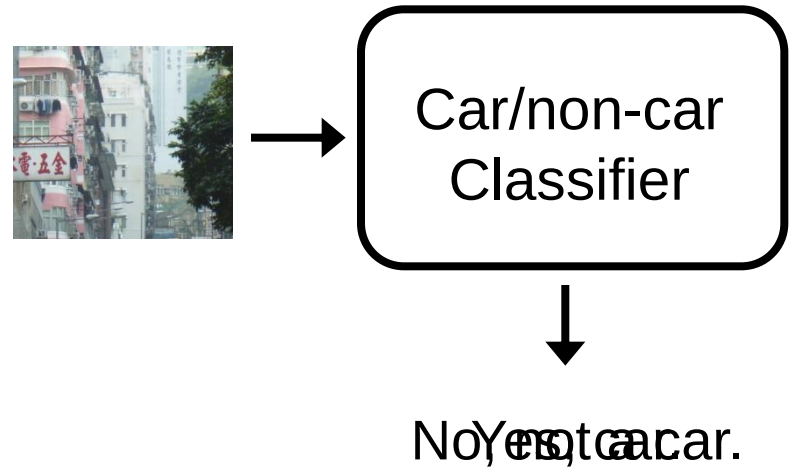
Topics of This Lecture

- Object Recognition and Categorization
 - Problem Definitions
 - Challenges
- Sliding-Window based Object Detection
 - Detection via Classification
 - Global Representations
 - Classifier Construction



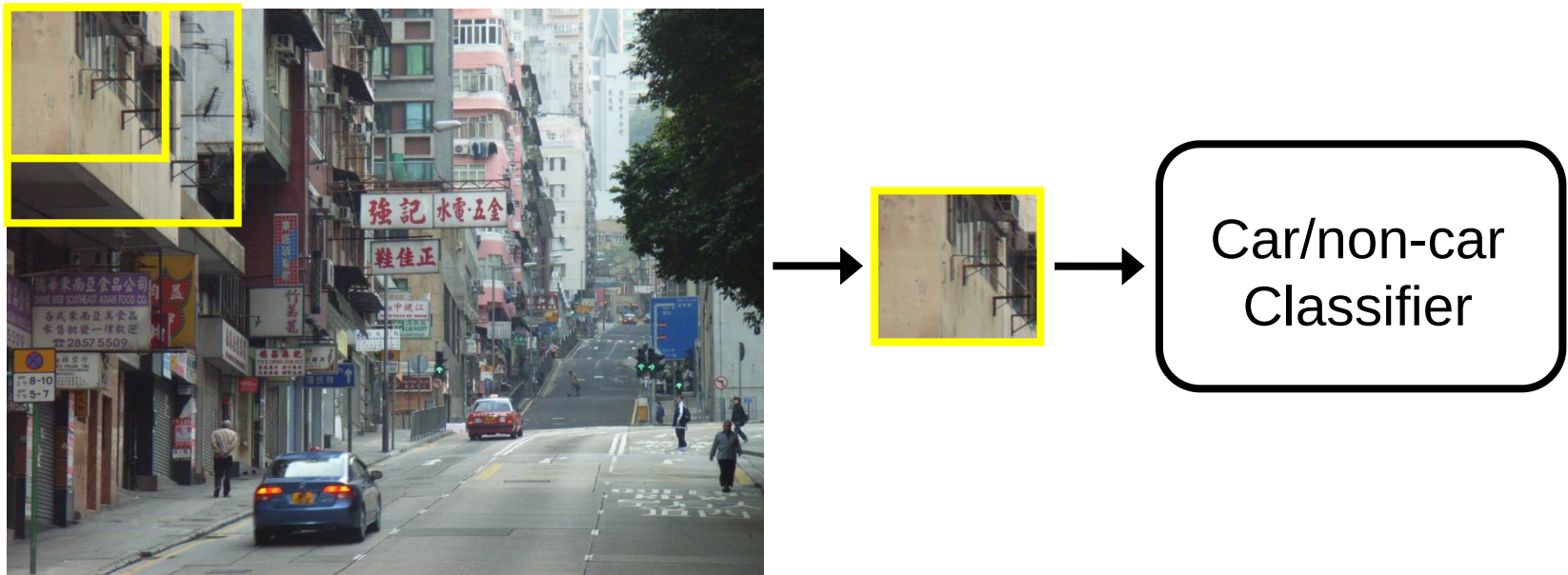
Detection via Classification: Main Idea

- Basic component: a binary classifier



Detection via Classification: Main Idea

- If the object may be in a cluttered scene, slide a window around looking for it.



- Essentially, this is a brute-force approach with many local decisions.

What is a Sliding Window Approach?

- Search over space and scale

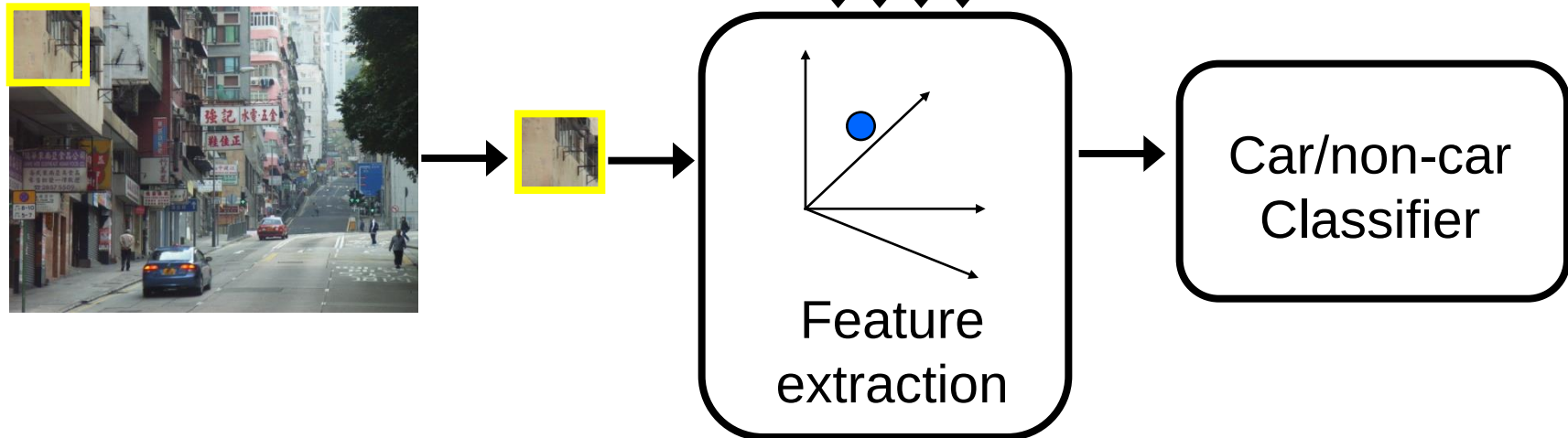
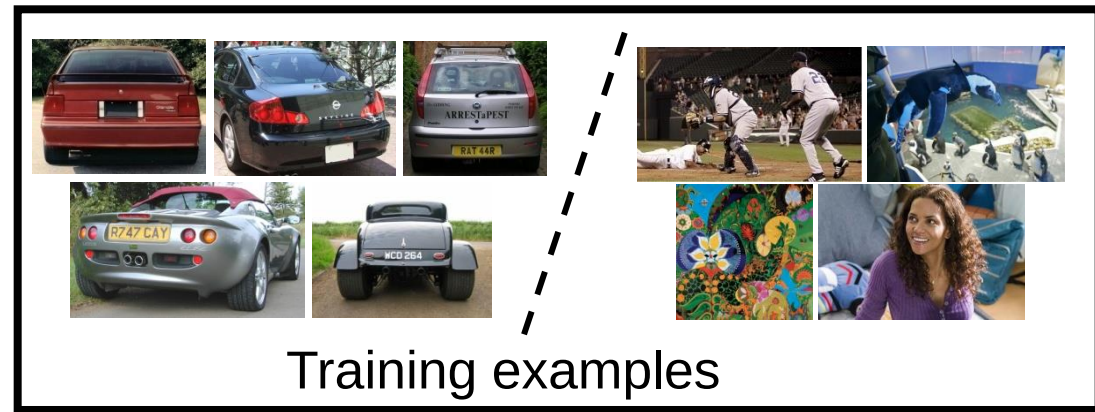


- Detection as subwindow classification problem
- *“In the absence of a more intelligent strategy, any global image classification approach can be converted into a localization approach by using a sliding-window search.”*

Detection via Classification: Main Idea

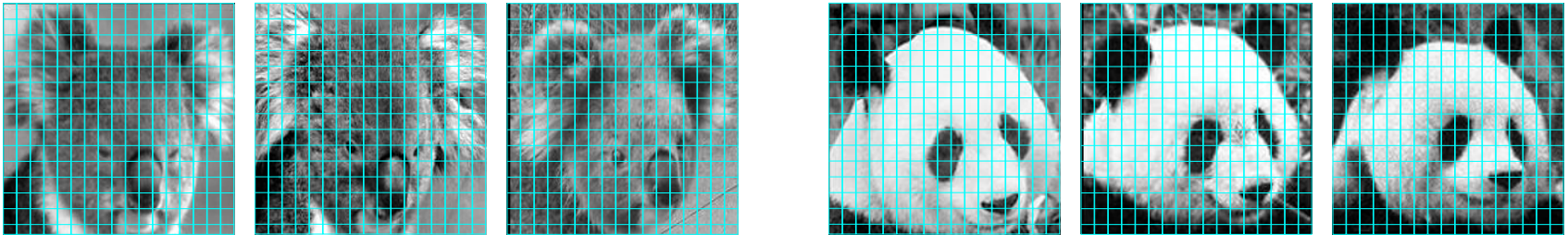
Fleshing out this pipeline a bit more, we need to:

1. Obtain training data
2. Define features
3. Define classifier



Feature Extraction: Global Appearance

- Pixel-based representations are sensitive to small shifts



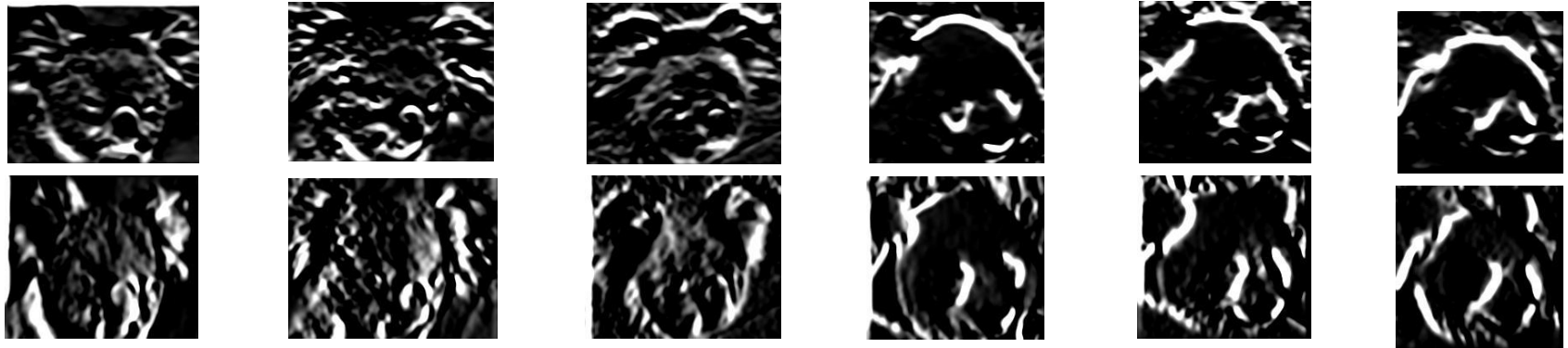
- Color or grayscale-based appearance description can be sensitive to illumination and intra-class appearance variation



Cartoon example:
an albino koala

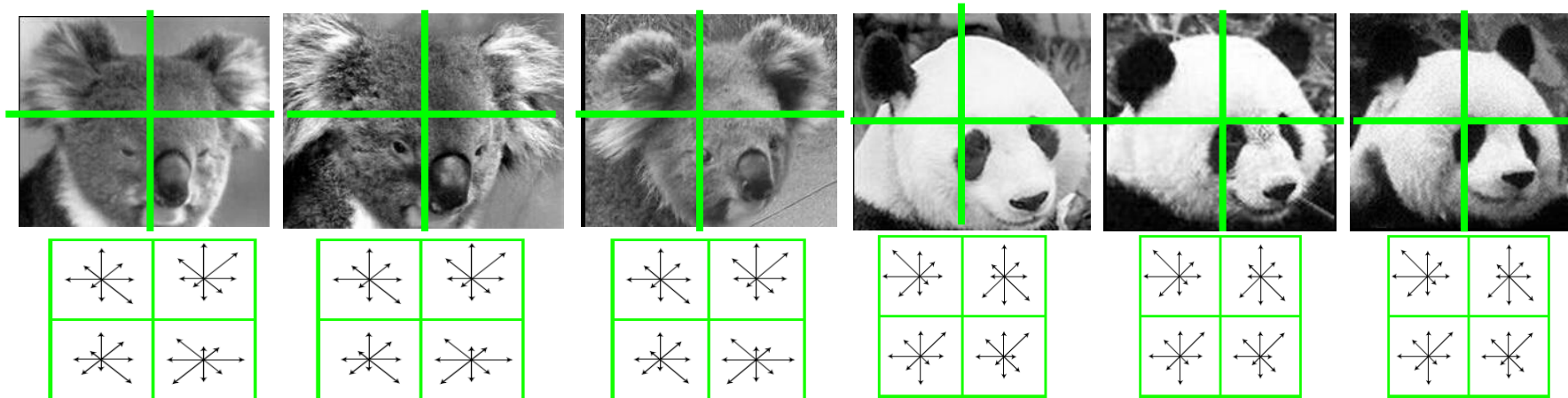
Gradient-based Representations

- Idea
 - Consider edges, contours, and (oriented) intensity gradients



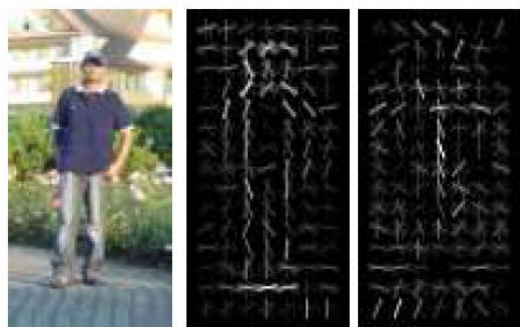
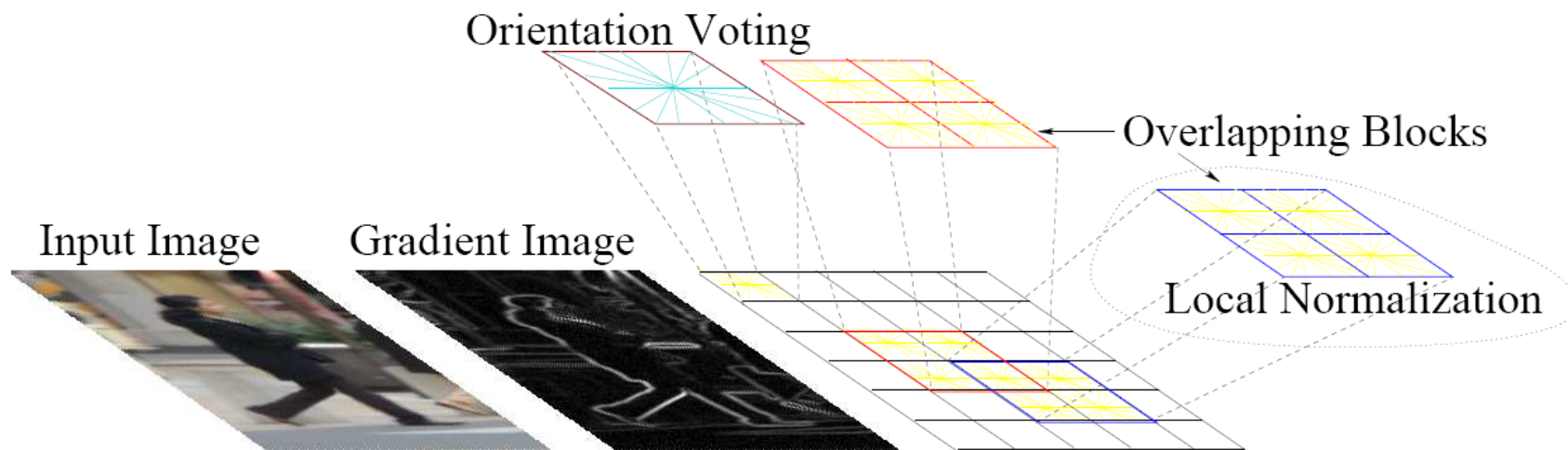
Gradient-based Representations

- Idea
 - Consider edges, contours, and (oriented) intensity gradients



- Summarize local distribution of gradients with histograms
 - Locally orderless: offers invariance to small shifts and rotations
 - Localized histograms offer more spatial information than a single global histogram (tradeoff invariant vs. discriminative)
 - Contrast-normalization: try to correct for variable illumination

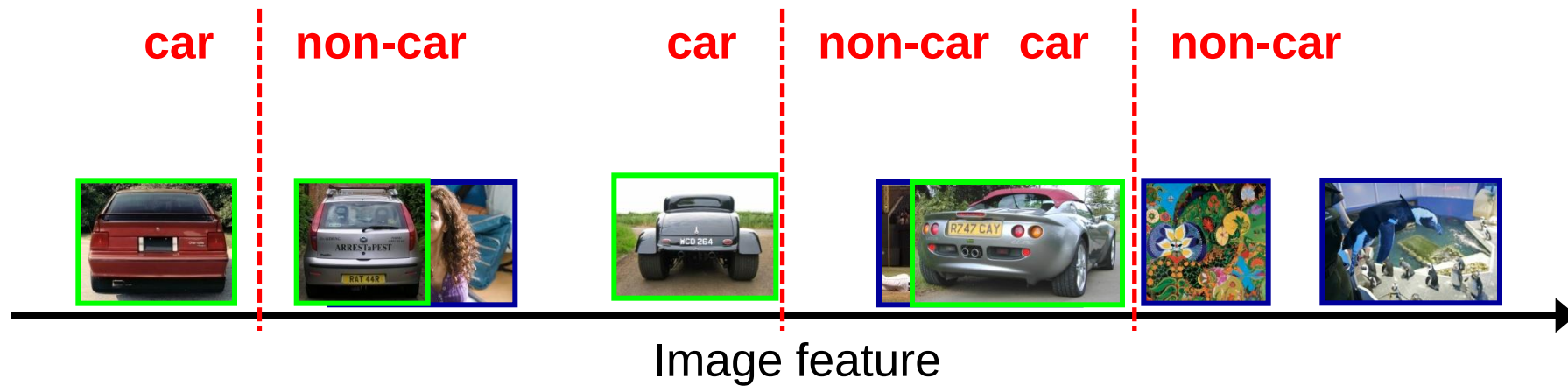
Gradient-based Representations: Histograms of Oriented Gradients (HoG)



- Map each grid cell in the input window to a histogram counting the gradients per orientation.
- Code available:
<http://pascal.inrialpes.fr/soft/olt/>

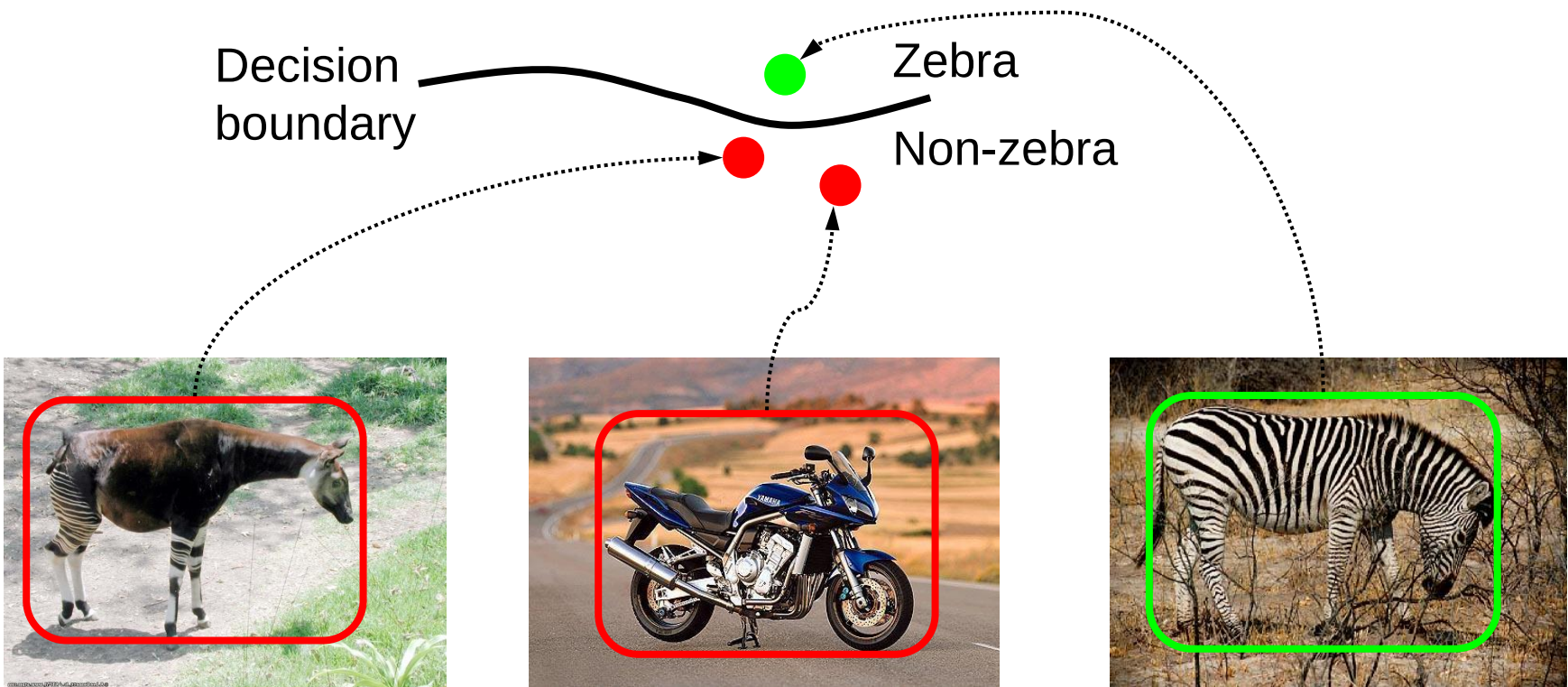
Classifier Construction

- How to compute a decision for each subwindow?



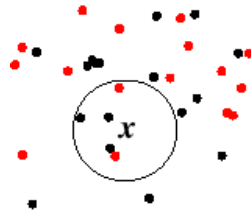
Discriminative Methods

- Learn a decision rule (classifier) assigning image features to different classes



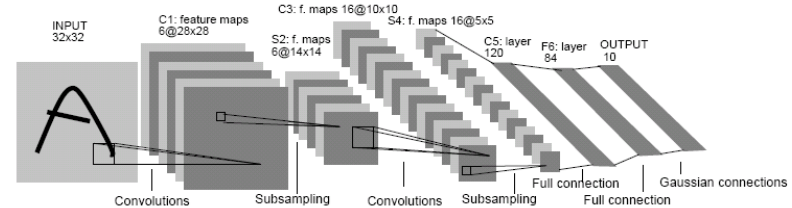
Classifier Construction: Many Choices...

Nearest Neighbor



Berg, Berg, Malik 2005,
Chum, Zisserman 2007,
Boiman, Shechtman, Irani 2008

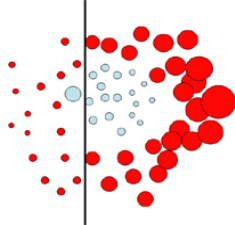
Neural networks



LeCun, Bottou, Bengio, Haffner 1998
Rowley, Baluja, Kanade 1998

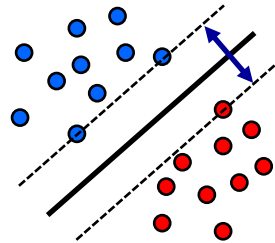
More on that next lecture...

Boosting



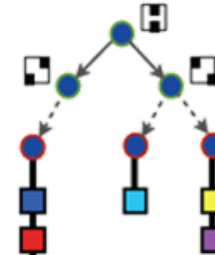
Viola, Jones 2001,
Torralba et al. 2004,
Opelt et al. 2006,
Benenson 2012, ...

Support vector machines



Vapnik, Schölkopf 1995,
Papageorgiou, Poggio '01,
Dalal, Triggs 2005,
Vedaldi, Zisserman 2012

Randomized Forests



Amit, Geman 1997,
Breiman 2001,
Lepetit, Fua 2006,
Gall, Lempitsky 2009,...

References and Further Reading

- A gentle introduction to Graph Cuts can be found in the following paper:
 - Y. Boykov, O. Veksler, [Graph Cuts in Vision and Graphics: Theories and Applications](#). In *Handbook of Mathematical Models in Computer Vision*, edited by N. Paragios, Y. Chen and O. Faugeras, Springer, 2006.
- Read how the interactive segmentation was first realized in MS Office 2010
 - C. Rother, V. Kolmogorov, Y. Boykov, A. Blake, [Interactive Foreground Extraction using Graph Cut](#), Microsoft Research Tech Report MSR-TR-2011-46, March 2011
- Try the GraphCut implementation at <https://pub.ist.ac.at/~vnk/software.html>